BERT shapes | Name every axis

B = batch size; L = padded token length; H = hidden width; C = number of classes; V = vocabulary size. Memorize these relationships, then inspect actual values.

Executed example: BERT-tiny, B=2, L=10, H=128, C=2, V=30,522. Texts: "Hello world." and "This example contains a few more words." Token counts are 5 and 10; the shorter sequence is padded.

Tensor / object	General shape	Actual example	Meaning
input_ids	[B,L]	[2,10]	long token indices
attention_mask	[B,L]	[2,10]	1 actual token; 0 padding
token_type_ids	[B,L]	[2,10]	BERT segment IDs; model dependent
Word embedding weights	[V,H]	[30522,128]	Learned lookup table
Word / combined embeddings	[B,L,H]	[2,10,128]	One H-vector per token
last_hidden_state	[B,L,H]	[2,10,128]	Contextual token representations
pooler_output / dropout	[B,H]	[2,128]	One sequence vector per example
classifier.weight	[C,H]	[2,128]	Linear layer stores output-first weights
classifier.bias	[C]	[2]	One bias per class
logits / probabilities	[B,C]	[2,2]	Raw scores / separately computed probabilities
labels / predictions	[B]	[2]	long class IDs / argmax outputs
cross-entropy loss	[]	()	Scalar with mean reduction
classifier.weight.grad	[C,H]	[2,128]	Gradient matches its parameter shape

nn.Linear(H,C) computes X @ weight.T + bias: [B,H] times [H,C] gives [B,C]. Reducing [B,L,H] over H instead leaves [B,L], which is not this head's expected feature width.

At B=1: logits [1,2], class labels [1], scalar loss (). Labels [1,1] need a deliberate trailing-dimension removal; unrestricted squeeze() would produce a scalar. This pack uses integer class targets, not the separate same-shape floating probability-target interface.

```
python shape_lab.py
python shape_lab.py --pdb
```

H=128 is specific to this tiny model. Other BERT models can use different widths. L changes with text and padding. Token IDs are integer indices; learned representations and weights are floating point.

Training state | Inspect the boundary

Line / action	What changes
zero_grad(set_to_none=True)	Managed parameter gradient buffers become None; weights stay unchanged.
forward + loss	Outputs and an autograd graph are created; this model's weights stay unchanged.
loss.backward()	Connected leaf parameter gradients accumulate. Weights are not updated.
optimizer.step()	Managed parameters and optimizer state update using stored gradients.

Ask separately: Is it trainable? Was its forward tracked? Is .grad present? Is the same parameter object managed? Did the live weight change? None can mean not calculated, cleared, unused, or frozen; it is different from a stored zero gradient.

Parameter scopes depend on the wrapper

In shape_lab.py	Scope
model.parameters()	Whole BertForSequenceClassification model.
model.bert.parameters()	Encoder and pooler, excluding classifier head.
model.classifier.parameters()	Only the head. In ML folders use model.backbone.classifier.

In the ML folders, model.backbone is the entire Hugging Face classifier, including its head. Do not assume a field named backbone always means encoder only. Inspect the registered modules.

Reliable snapshots; location of detach matters

```
parameter = model.classifier.weight
before = parameter.detach().clone()
# Run forward, loss, backward and optimizer.step().
delta = (parameter.detach() - before).abs().max().item()
```

detach disconnects autograd but shares storage. detach().clone() creates a disconnected independent snapshot. detach is not a CPU transfer. Detaching the loss before backward breaks training; detaching a copy for logging or measurement is useful. eval() does not disable gradients or freeze parameters.

breakpoint() in a few lines

Insert breakpoint() before the suspect operation. p EXPR prints; n next; s step into; c continue; q quit. l nearby source; w stack; u/d change stack frames. If collation fails before a model pause, put the breakpoint inside the collator.

```
p type(logits), logits.shape, logits.dtype, logits.device
p labels.shape, labels.dtype
p loss.requires_grad, loss.grad_fn
p model.classifier.weight.grad
p optimizer.param_groups[0]["lr"]
```

Those paths match shape_lab.py. In an ML wrapper, inspect model.backbone.classifier.weight.grad. Variables exist only after their assignment has executed. User code can return a tensor or a ModelOutput; inspect the type before assuming .logits is needed.

Small patterns | Recognize, then look up

Problem wording	Useful tool	Edge case to test
Last matching review	Overwrite in a scan; or reversed scan	A later draft; no match; list order vs time.
Latest timestamp	Comparison / max with key	Equal timestamps; negative values.
Already seen this ID?	set for membership	Keep an output list if order matters.
Value associated with ID	dict lookup / join	Missing ID versus valid label zero.
All texts per category	defaultdict(list) / setdefault	Separate lists; keep duplicates and order.
Modify a string	Slices / literal replace	Strings are immutable; replacement count.
Custom ordering	sorted with a key	Numeric vs lexicographic; stable ties.
Missing / null / blank	Membership, identity, type checks	False, 0 and empty containers may be valid.
One-to-many records	Nested scan / flatten	Nonblank paragraphs and label zero.
Independent file exports	Reset lookup inside outer loop	Same ID in two files; no state leakage.

When the API is not in your memory

Restate the contract and one edge case. Identify the operation. Search the official docs for the small operation, not the whole problem. Inspect its signature and return type. Run a two-line REPL experiment, then apply and test it.

```
print("a.b a.b".replace("a.b", "X", 1))
print(bool(0), 0 == False, type(False) is int)
```

Example searches: "Python str.replace count", "Python sorting stable key tuple", "dict missing None sentinel", "PyTorch detach shares storage". Full links and lookup prompts for every Python question are in LOOKUP_PLAYBOOK.md.

Python types and strings

https://docs.python.org/3/library/stdtypes.html

PyTorch detach / storage

https://docs.pytorch.org/docs/2.7/generated/torch.Tensor.detach.html

Practice structure

Six Python folders x three pure-function problems = 18. Five ML folders x three sequential defects = 15. Work one folder at a time. Try, predict, inspect, request one hint, repair, explain, then replay. A complete boilerplate template is supplied so you can focus on these operations.

The code and shape example were executed on Linux CPU with PyTorch 2.7.1 and transformers 4.57.1. Exact model sizes and loss decimals vary across models/versions. See VERIFICATION.md for tested checks and platform limits.